

PulseDrive

SYSTEM DESIGN DOCUMENT

AI-Powered Predictive Vehicle Health Monitoring System
Qualcomm Snapdragon Hackathon

Member	Role
Tanish Aggarwal	IoT & Machine Learning
Ishaan Maheshwari	IoT
Anshuman Dutta	Agentic AI
Yash Goel	Development

Document Version 1.0

1. Overview

PulseDrive is a real-time, AI-powered predictive vehicle health monitoring system. It continuously collects telemetry from an onboard sensor rig, streams it to a backend over a live WebSocket channel, computes a health score and diagnosis, and mirrors predictions on-device using two LiteRT models running directly on an Android phone.

1.1 Problem Statement

Vehicle failures — overheating, tyre imbalance, motor overload, or a fire hazard — are typically detected only after they become visible or physically noticeable. Standard dashboards display raw sensor values without interpreting them, leaving drivers and fleet operators without early warning of developing faults.

1.2 Design Goals

- Real-time telemetry with sub-second latency between sensor and dashboard.
- On-device inference so critical predictions do not depend on network availability.
- A single source of truth (MongoDB) for historical telemetry and trend analysis.
- A backend that scales by vehicle count without becoming an inference bottleneck.
- A schema that stays stable across hardware, backend, and both client apps.

2. System Architecture

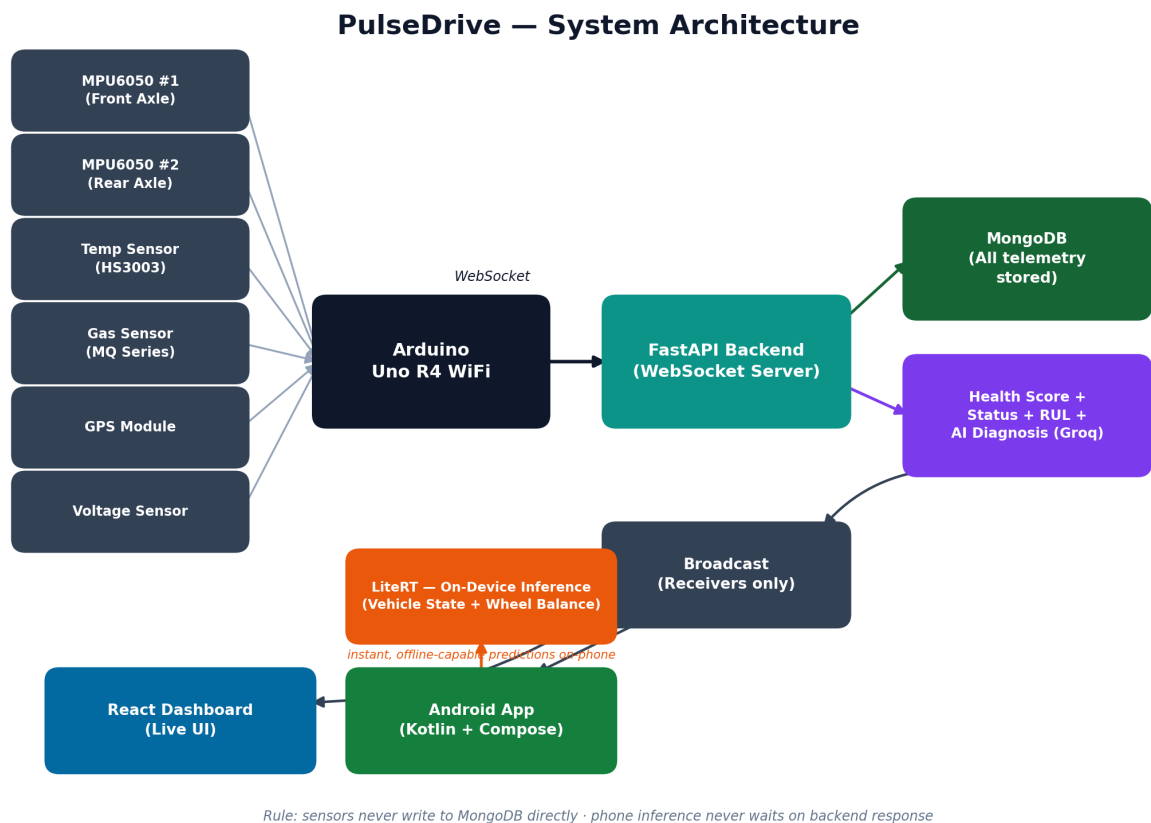


Figure 1 — End-to-end system architecture

2.1 Component Responsibilities

Component	Responsibility
Arduino Uno R4 WiFi	Reads all sensors, packages a JSON payload, publishes it over WebSocket.
FastAPI Backend	Validates incoming packets, computes health metrics, stores data, broadcasts to receivers.
MongoDB	Persists every telemetry packet for history and future trend graphs.
React Dashboard	Renders live gauges, charts, GPS, and AI diagnosis from the WebSocket feed.
Android App (Kotlin)	Subscribes to live telemetry and runs on-device LiteRT inference.
LiteRT Models	Classify vehicle motion state and wheel balance directly on-device.

Key rule: sensors never write to MongoDB directly, and the phone never blocks on a backend response to know whether a wheel is imbalanced — that decision is made locally, on-device, in milliseconds.

3. Backend Data Flow

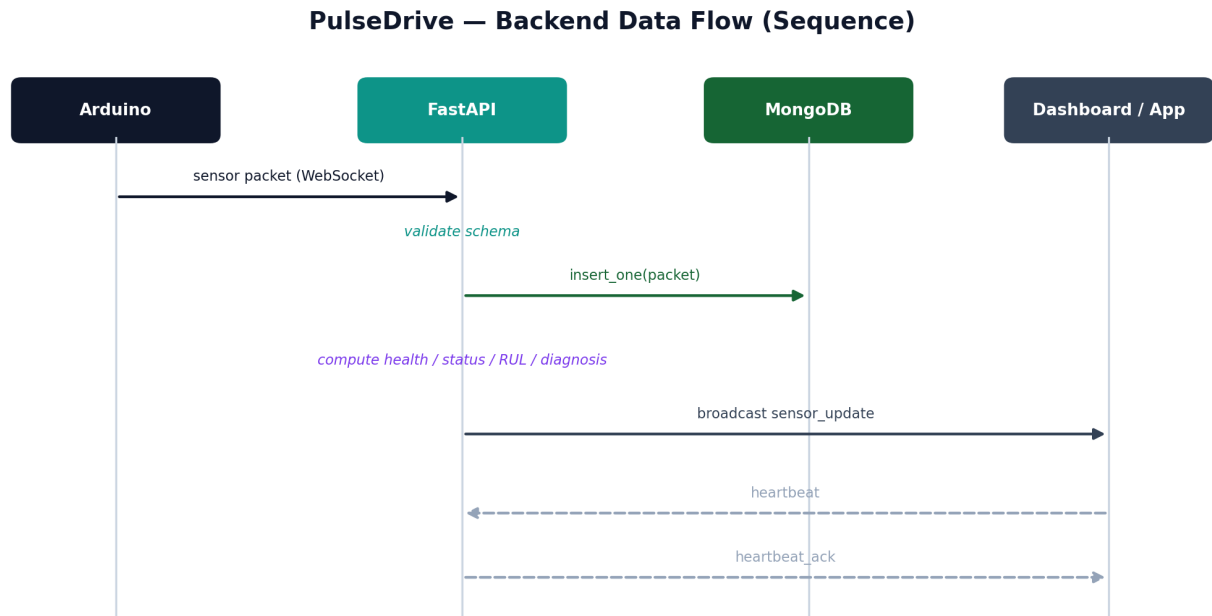


Figure 2 — Sequence of events for a single telemetry packet

3.1 Step-by-Step

- 1 Arduino publishes a JSON sensor packet over the /ws/live WebSocket.
- 2 FastAPI validates the packet against the expected schema; malformed packets are logged and dropped.
- 3 The packet is inserted into MongoDB — every packet, not just anomalous ones.
- 4 The backend computes health score, status, remaining useful life (RUL), and an AI diagnosis.
- 5 The enriched sensor_update payload is broadcast to all receiver connections (dashboard, Android app) — never back to the sender.
- 6 Idle connections are kept alive via heartbeat / heartbeat_ack, which are not persisted to MongoDB.

3.2 Architecture Evolution

The pipeline went through three iterations before reaching its current form: an initial HTTP-polling design (ESP32 → FastAPI → MongoDB → Android polling), an MQTT + Mosquitto broker design evaluated for its native multi-subscriber support, and the current WebSocket design, which was retained for the hackathon because it provides one simple real-time channel to both clients without the operational overhead of running a separate broker process.

4. Database Schema (MongoDB)

A single collection stores every telemetry packet. The schema below is shared verbatim across the Arduino payload, the FastAPI validation model, and both client apps.

```
{
  "vehicleId": "CAR001",
  "timestamp": "2026-07-19T10:30:00Z",
  "temperature": 32.4,
  "voltage": 12.4,
  "gasSensor": { "value": 220, "unit": "ppm" },
  "gps": { "lat": 28.6139, "lng": 77.2090 },
  "mpu1": { "accX": 0.2, "accY": 0.1, "accZ": 9.8,
            "gyroX": 0.1, "gyroY": 0.2, "gyroZ": 0.3 },
  "mpu2": { "accX": 0.3, "accY": 0.2, "accZ": 9.7,
            "gyroX": 0.1, "gyroY": 0.3, "gyroZ": 0.2 },
  "healthScore": 92,
  "status": "Healthy",
  "remainingUsefulLife": "95%",
  "aiDiagnosis": "All readings within normal range.",
  "recommendation": "No maintenance required."
}
```

4.1 Outgoing WebSocket Envelope

Broadcast messages wrap the payload above inside a standardized envelope so clients can distinguish telemetry from control messages such as heartbeats:

```
{
  "type": "sensor_update",
  "source": "arduino",
  "vehicleId": "CAR001",
  "received_at": "2026-07-19T10:30:00.412Z",
  "payload": { ... schema above ... }
}
```

5. On-Device AI Models

Both models are int8-quantized TensorFlow Lite (LiteRT) models bundled into the Android app's assets, and both run entirely on-device via the phone's NPU — no network call is made for either prediction.

	Vehicle State Model	Wheel Imbalance Model
File	vehicle_state_model_int8.tflite	wheel_imbalance_model_int8.tflite
Input shape	[1, 6]	[1, 3]
Input features	Accel X, Y, Z · Gyro X, Y, Z	Gyro X, Y, Z
Output classes	Stationary / Moving / Inclined / Declined	Balanced / Imbalance
Observed confidence	~0.996 (test rig)	~0.996 (test rig)

Note: confidence values above were observed on a static test rig. Real driving data should be used to re-validate both models before treating these numbers as production-representative.

5.1 Why On-Device (Edge) Inference

	Cloud-only	PulseDrive (Edge)
Latency	High (network round trip)	Milliseconds
Works offline	No	Yes
Battery cost	Network-heavy	NPU-efficient
Scales across vehicles	Backend inference bottleneck	Backend only stores + summarizes

We evaluated Qualcomm's Track 1 (LiteRT-LM / Gemma) against Track 2 (LiteRT / TFLite / classical ML) and deliberately chose Track 2 — our problem is a classification task over IMU data, not a language task, so an LLM would add cost without improving accuracy. Snapdragon X Elite's dedicated NPU is what makes running two always-on models on a phone battery-realistic, and the same NPU access on a Copilot+ PC allowed local model validation during development, with Qualcomm GenIE integration left open as a future upgrade path.

6. Vehicle Health Scenarios

The backend's health-scoring logic maps sensor ranges to five distinct operating scenarios, each with its own dashboard theme so status is visible at a glance.

Status	Trigger Condition	Health Score	Theme
Healthy	Normal temp/voltage/smoke, stable MPU readings	95–100	Green pulse
Warning	Tyre imbalance — MPU1 and MPU2 vibration diverge	70–80	Yellow glow
Critical	Overheating (90–120°C)	30–45	Orange pulse
Critical	Motor overloading — low voltage + sustained heat	45	Orange pulse
Emergency	Smoke 200–500 ppm with high temperature	10	Flashing red

7. Hardware Layout

Component	Purpose
Arduino Uno R4 WiFi (or UNO Q)	Main controller — reads all sensors, publishes telemetry
MPU6050 x2	Accelerometer + gyroscope, mounted on front and rear axle
HS3003	Temperature / humidity, mounted near engine/motor housing
MQ Gas Sensor	Smoke / gas concentration (ppm), mounted near engine bay
GPS Module	Location telemetry over UART
BTS7960	Motor driver for DC motor control
Voltage Sensor	Battery voltage via divider circuit

Why two IMUs? A healthy vehicle shows similar vibration signatures at both axles. An imbalanced wheel produces a measurable difference between the two — a single IMU cannot capture that comparison.

8. Technology Stack

Layer	Technology
Firmware	Arduino (C++), WebSocket client library
Backend	Python, FastAPI, WebSockets, Pydantic, Groq (diagnosis text), JWT auth
Database	MongoDB
Web Frontend	React, TypeScript, Vite, Tailwind CSS, Chart.js
Mobile App	Kotlin, Jetpack Compose, MVVM, Retrofit / OkHttp WebSocket, Hilt
On-Device AI	TensorFlow Lite / LiteRT (int8 quantized models)
Target Hardware	Snapdragon X Elite (NPU), Copilot+ PC for local dev/validation

9. Known Limitations

- GPS accuracy is unreliable indoors — expected behavior, not a defect.
- The Groq-based diagnosis text occasionally requires a schema retry on malformed JSON output.
- Model confidence figures are from static test-rig conditions and need re-validation on real driving data.
- Wiring diagram / physical rig photos are maintained separately in the project README.